

Project Report

A Computer Vision Project on

Facial Emotion Recognition

Submitted by

Kalpita Sharma(M24DE3042), Meenal Soni (M24DE3047),
Atiksha Soni (M24DE3020), Dhanashree Hood(M24DE3028),
Zarna Palera(M24DE3088)

Under the Supervision
Of
Professor Pratik Majumdar



Department of Artificial Intelligence
Indian Institute of Technology Jodhpur

Team Composition

- Kalpit Sharma(M24DE3042)
- Meenal Soni (M24DE3047)
- Atiksha Soni (M24DE3020)
- Dhanashree Hood(M24DE3028)
- Zarna Palera(M24DE3088)

Project Link: <https://57e9-13-232-194-50.ngrok-free.app/>
<https://57e9-13-232-194-50.ngrok-free.app/>

Project Code:

<https://huggingface.co/kalpitsharma/face-emotion-recognition/tree/main>

Index

1. Introduction
2. Objectives
3. Dataset
4. Technology
5. Methodology
6. Model Performance Analysis
7. Advantages and Disadvantages
8. Conclusion
9. References

Introduction

Emotion is an essential and intrinsic part of human cognition and social interaction. It plays a pivotal role in shaping our perceptions, guiding our actions, and influencing our relationships. Facial expressions, being one of the most universal and visible indicators of emotional states, offer a rich source of information that can be harnessed by machines to better understand human behavior.

With the rapid advancement of artificial intelligence (AI) and computer vision, automated emotion detection has become a significant research area with real-world impact across various domains. Unlike textual or verbal communication, facial expressions can convey subtle, complex, and spontaneous emotional cues in real-time. Leveraging this through emotion-aware systems leads to more natural and adaptive human-computer interactions.

Applications of Emotion Detection:

1. **Human-Computer Interaction (HCI):** Emotionally intelligent systems can adapt their responses based on the user's emotional state, improving usability and user experience in applications such as virtual assistants, chatbots, and gaming.
2. **Mental Health Monitoring:** Emotion detection tools can help track mood variations in individuals over time, supporting early diagnosis of mental health disorders such as depression, anxiety, or bipolar disorder.
3. **Driver Safety Systems:** Real-time facial emotion analysis can help detect driver fatigue, stress, or distraction, triggering alerts to prevent accidents and promote road safety.
4. **E-learning and Education:** Emotion-aware e-learning platforms can adapt teaching styles or difficulty levels based on a learner's engagement and frustration levels, creating personalized learning environments.
5. **Customer Feedback and Marketing:** Analyzing emotional reactions from facial

expressions during product testing, advertisements, or customer service interactions provides valuable insights for improving products and services.

Objectives

The primary aim of this project is to develop and evaluate multiple computational approaches for detecting emotions from facial expressions using image data. Emotion recognition, being a nuanced and multifaceted task, benefits from a combination of traditional and modern machine learning paradigms. The key objectives of the project are outlined below:

1. To understand and implement emotion detection using facial images

This objective focuses on gaining a comprehensive understanding of how facial expressions can be mapped to distinct emotional states using computational models. It involves:

- Studying the theoretical background of emotion representation, including the psychology behind basic emotions (e.g., Ekman's six universal emotions).
- Understanding how facial muscle movements correspond to different emotions and how they are reflected in 2D grayscale images.
- Implementing various data preprocessing techniques such as grayscale conversion, normalization, face alignment, and resizing to prepare the input for machine learning and deep learning models.
- Exploring different strategies for labeling and categorizing emotions and assessing how well models generalize across these labels.

2. To compare traditional machine learning models with deep learning models in terms of accuracy, training complexity, and interpretability

This objective involves a detailed comparison between classical machine learning models and modern deep learning approaches, focusing on:

- **Accuracy:** How well each model can classify emotions on unseen test data. Evaluation metrics such as precision, recall, F1-score, and confusion matrices are used.
- **Training Complexity:** Understanding computational costs, including time to train, memory usage, model tuning complexity, and scalability to larger datasets.
- **Interpretability:** Assessing how explainable each model is. While traditional models like Decision Trees and Logistic Regression are easier to interpret, deep learning models (especially CNNs and Transformers) are often considered black boxes.

3. To create a pipeline that can be adapted for real-time emotion detection applications

The final objective is to design a modular and efficient pipeline that can be deployed in real-world, real-time scenarios. This includes:

- Building a robust preprocessing pipeline for real-time image capture, face detection (e.g., using OpenCV's Haar cascades or Dlib), and image normalization.
- Developing a model inference system optimized for speed and responsiveness, possibly with techniques like model pruning or quantization for deployment on edge devices.
- Ensuring modularity so that components (e.g., preprocessing, model loading, inference) can be swapped or improved independently.
- Testing the pipeline under simulated real-time conditions, such as video streams or webcam input, to assess latency and frame-wise accuracy.

Dataset: FER2013

Dataset Overview

- Source: <https://www.kaggle.com/datasets/msambare/fer2013>
- Classes (Emotions):
 1. Angry

2. Disgust
3. Fear
4. Happy
5. Sad
6. Surprise
7. Neutral

Preprocessing Steps

- Conversion to grayscale
- Image resizing to 48x48
- Normalization (pixel values scaled to [0, 1])
- One-hot encoding of labels
- Data augmentation (rotation, zoom, flip, brightness) for deep learning models

Technology

To build a robust and efficient facial emotion recognition system, a variety of tools, libraries, and frameworks were employed throughout different stages of the machine learning and deep learning pipeline. These components were selected based on their relevance to image processing, model training, visualization, and ease of development.

Technical Architecture Components:

1. Face Detection:
 - Uses Haar Cascade Classifier (haarcascade_frontalface_default.xml)
 - Real-time face detection through webcam feed (camera.py)
2. Web Interface:

- Flask-based web application (app.py)
- HTML templates for different model interfaces
- Static resources for frontend

Programming Language: Python 3.x

Python was the primary language used for this project due to its simplicity, readability, and the extensive ecosystem it offers for scientific computing and machine learning.

Why Python?

- Readable syntax suitable for rapid prototyping
- Rich ecosystem for machine learning and data processing
- Seamless integration with visualization and deployment tools

Image Processing Libraries: OpenCV, PIL

Image processing plays a vital role in preprocessing facial images before feeding them into learning models.

- OpenCV (Open Source Computer Vision Library):
 - Used for face detection (e.g., Haar cascades), image resizing, and real-time video handling.
 - Supports a wide range of image transformations (blurring, thresholding, edge detection).
 - Offers GPU acceleration for faster computation.
- PIL (Python Imaging Library) / Pillow:
 - Used for basic image operations such as resizing, format conversion, and image augmentation.
 - Simplifies loading and manipulating image files during preprocessing.

Machine Learning Models: Scikit-learn

Scikit-learn is one of the most powerful libraries for implementing classical machine learning algorithms in Python.

Key Uses in the Project:

- Implemented models like KNN, SVM, Logistic Regression, and Random Forest.
- Feature extraction techniques such as PCA and standardization.
- Model evaluation using metrics like confusion matrix, accuracy score, and cross-validation.

Deep Learning Framework: TensorFlow (with Keras API)

Deep learning models were developed using TensorFlow, with the high-level Keras API for ease of model building, training, and evaluation.

Why TensorFlow/Keras?

- Flexible architecture that supports CNNs, RNNs, and Transformer models
- Built-in support for GPU acceleration and distributed training
- Intuitive syntax for defining models, loss functions, optimizers, and callbacks

Development Environment

Both Jupyter Notebooks were utilized for interactive development and training.

Dependencies

- Python-based implementation
- Key libraries: TensorFlow, scikit-learn, OpenCV
- Web framework: Flask
- Frontend: HTML, CSS, JavaScript

Methodology

To approach the problem of facial emotion detection, we experimented with a variety of models categorized into three methodological domains: traditional machine learning, deep learning, and transfer learning.

5.1 Traditional Machine Learning Approaches

Traditional ML methods rely on manually engineered features extracted from images. These features attempt to capture shape, texture, or intensity variations that correspond to different emotional expressions.

(a) K-Nearest Neighbors (KNN)

- Feature Engineering:
 - Histogram of Oriented Gradients (HOG): Captures gradient orientation in localized portions of the image, highlighting edges and facial structures (e.g., eyebrows, mouth curvature).
- Working Principle: A lazy learner that stores all training data and makes predictions based on the k most similar data points.
- Hyperparameters:
 - Value of k : tuned using cross-validation (typically 3, 5, or 7).
 - Distance metric: Euclidean distance.

(b) Support Vector Machine (SVM)

- Preprocessing:
 - Principal Component Analysis (PCA): Reduced feature space for efficient training and to avoid the curse of dimensionality.
 - Local Binary Patterns (LBP): Encodes texture by thresholding pixel neighborhoods. Effective for recognizing micro-expressions.
- Model Configurations:

- Kernels tested: Linear and RBF (Radial Basis Function).
- RBF provided better results due to non-linear separation boundaries.

(c) Random Forest Classifier

- Model Architecture:
 - An ensemble of decision trees trained on different bootstrapped samples of the data.
 - Uses majority voting for final prediction.

(d) Logistic Regression

- Purpose:
 - Serves as a baseline model to benchmark more complex methods.
- Input:
 - Raw pixel intensities or reduced PCA features.

5.2 Deep Learning Approaches

Deep learning models automatically learn hierarchical features from raw image pixels without requiring manual feature extraction.

(a) Convolutional Neural Networks (CNN)

- Architecture:
 - Input: 48×48 grayscale image
 - Layers:
 - Multiple convolutional layers with ReLU activation
 - MaxPooling layers to reduce spatial dimensions
 - Dropout layers to prevent overfitting
 - Fully connected Dense layers for final classification
- Training Details:

- Loss function: Categorical Crossentropy
- Optimizer: Adam (adaptive learning rate)
- Epochs: 30-50
- Batch size: 32 or 64
- Data Augmentation:
 - Rotation, horizontal flipping, brightness variation
 - Used to improve generalization and reduce overfitting

(b) CNN + RNN Hybrid

- Rationale:
 - Emotions in videos evolve over time; combining CNN with RNN can model temporal dynamics.
- Pipeline:
 - CNN extracts a feature vector from each image/frame
 - RNN processes the sequence of features to capture temporal patterns

(c) Vision Transformer (ViT-B/32 with CLIP)

- Objective:
 - Leverage attention mechanisms to focus on discriminative regions of the face.
- Architecture:
 - Uses the Vision Transformer architecture (ViT-B/32)
 - Implements CLIP (Contrastive Language-Image Pre-training) for improved feature extraction
 - Fully connected layers for classification

5.3 Custom Weighted Model

- A custom ensemble approach that combines predictions from multiple models

- Weights assigned based on individual model performance
- Designed to leverage the strengths of different architectures

Model Performance Analysis

1. Support Vector Machine (SVM)

- Overall Accuracy: 45%
- Best performing emotions:
 - Happy (F1-score: 0.63)
 - Surprise (F1-score: 0.55)
- Weighted Average:
 - Precision: 0.44
 - Recall: 0.45
 - F1-score: 0.44

2. Random Forest

- Overall Accuracy: 45%
- Best performing emotions:
 - Happy (F1-score: 0.60)
 - Surprise (F1-score: 0.58)
- Notable performance:
 - Perfect precision for disgust (1.00)
- Weighted Average:
 - Precision: 0.45
 - Recall: 0.45
 - F1-score: 0.42

3. K-Nearest Neighbors (k-NN)

- Overall Accuracy: 45%
- Best performing emotions:
 - Happy (F1-score: 0.62)
 - Surprise (F1-score: 0.51)
- Weighted Average:

- Precision: 0.44
- Recall: 0.45
- F1-score: 0.43

4. Logistic Regression

- Overall Accuracy: 44%
- Best performing emotions:
 - Happy (F1-score: 0.63)
 - Surprise (F1-score: 0.53)
- Weighted Average:
 - Precision: 0.43
 - Recall: 0.44
 - F1-score: 0.43

5. CNN Model

- Accuracy: 33.49%
- Precision: 0.339
- Recall: 0.334
- F1 Score: 0.326

6. CNN + RNN Hybrid Model

- Accuracy: 21.30%
- Precision: 0.167
- Recall: 0.213
- F1 Score: 0.129

7. Vision Transformer (ViT-B/32)

- Accuracy: 43.74%
- Precision: 0.487
- Recall: 0.437
- F1 Score: 0.431

8. Custom Weighted Model

- Accuracy: 45.77%
- Precision: 0.013
- Recall: 0.458
- F1 Score: 0.124

Key Findings

- Model Performance Comparison:
 - Traditional ML models showed consistent performance around 45% accuracy
 - Vision Transformer performed similarly to traditional ML models
 - CNN and CNN+RNN models showed lower performance compared to other approaches
- Emotion-wise Analysis:
 - "Happy" emotion consistently showed the best recognition across all models
 - "Disgust" had high precision but low recall in most models
 - "Fear" was generally more challenging to classify accurately
- Best Performing Model:
 - The Custom Weighted Model achieved the highest accuracy at 45.77%
 - Vision Transformer showed the most balanced performance across metrics

Advantages and Disadvantages

In machine learning and deep learning, no one model is universally optimal. Each algorithm or approach has trade-offs depending on factors such as dataset size, complexity of the task, need for interpretability, computational resources, and time

constraints.

7.1 Traditional Machine Learning Models

Traditional ML models like KNN, SVM, Random Forest, and Logistic Regression rely on manually engineered features and are generally simpler in architecture and training requirements.

K-Nearest Neighbors (KNN)

Advantages:

- **Simplicity:** Intuitive and easy to implement.
- **No Training Phase:** KNN is a lazy learner, requiring no training time; only stores the dataset.
- **Non-parametric:** Can model non-linear decision boundaries given sufficient data.

Disadvantages:

- **Computational Inefficiency at Inference:** Distance computation with all training points makes it slow at prediction time.
- **Memory-Intensive:** Needs to store the entire dataset.
- **Sensitive to Noise and Feature Scaling:** Performance can degrade with irrelevant or unscaled features.

Support Vector Machine (SVM)

Advantages:

- **Effective in High Dimensions:** Performs well in spaces with many features.
- **Robust to Overfitting (with regularization):** Maximizes the margin between classes for better generalization.
- **Kernel Trick:** Allows modeling complex, non-linear decision boundaries using functions like RBF.

Disadvantages:

- **High Training Time:** Especially with large datasets or when using non-linear kernels.

- **Parameter Sensitivity:** Performance highly depends on proper tuning of kernel parameters (e.g., C , γ).
- **Limited Scalability:** Difficult to scale to massive datasets due to computational complexity.

Random Forest

Advantages:

- **Handles Non-Linearity:** Ensemble learning captures complex decision surfaces.
- **Reduces Overfitting:** By averaging across many trees.
- **Feature Importance:** Provides insights into which features are most relevant.
- **Robustness:** Performs reasonably well without extensive tuning.

Disadvantages:

- **Less Interpretable:** While individual trees are interpretable, the full forest is not.
- **Can Overfit on Noisy Data:** Especially if too many deep trees are used.
- **Limited on Raw Image Data:** Relies heavily on high-quality feature extraction.

Logistic Regression

Advantages:

- **Interpretable:** Provides coefficients that indicate feature influence.
- **Efficient:** Fast training and low computational requirements.
- **Good for Linearly Separable Data:** Performs well in simple classification tasks.

Disadvantages:

- **Poor with Non-Linear Data:** Assumes a linear decision boundary.
- **Underperforms on Complex Features:** Not suitable for tasks like image recognition without preprocessing.

7.2 Deep Learning Models

Deep learning models learn hierarchical features directly from raw data. These models are particularly effective for tasks like image recognition, where spatial patterns are

crucial.

Convolutional Neural Network (CNN)

Advantages:

- Automatic Feature Learning: Learns to extract relevant spatial features from raw image data.
- Translation Invariance: Through pooling operations, CNNs can recognize patterns regardless of location.
- Highly Accurate: Outperforms traditional models in most visual recognition tasks.

Disadvantages:

- Data-Hungry: Requires large labeled datasets for training.
- Computationally Intensive: Demands GPUs for reasonable training time.
- Harder to Interpret: Internal representations are not always intuitive or transparent.

CNN + RNN (Hybrid Model)

Advantages:

- Spatio-Temporal Modeling: Combines spatial analysis (CNN) with temporal memory, ideal for video and time-series applications.
- Improved Accuracy: Especially when handling multi-frame or video data.

Disadvantages:

- Training Complexity: More layers and dependencies lead to longer training time and greater risk of overfitting.
- Requires Frame Sequences: Not well suited for static image datasets unless artificially adapted.
- Harder to Tune: More hyperparameters (e.g., sequence length, number of units).

Vision Transformer (ViT)

Advantages:

- Global Attention: Transformers allow the model to focus on the most relevant facial

regions, regardless of their position.

- **State-of-the-Art Performance:** Excels in capturing complex dependencies between spatial features.
- **Scalable to Larger Models:** More effective with large-scale datasets and computational resources.

Disadvantages:

- **Resource-Heavy:** Requires substantial computational power and memory.
- **Long Training Time:** Due to complexity and number of parameters.
- **Architectural Complexity:** Harder to design and debug compared to conventional CNNs.

7.3 Custom Weighted Model

Advantages:

- **Leverages Strengths of Multiple Models:** Combines predictions from different architectures.
- **Improved Accuracy:** Can achieve better results than individual models.
- **Adaptability:** Weights can be adjusted based on specific emotion recognition needs.

Disadvantages:

- **Increased Complexity:** Requires managing multiple models.
- **Higher Computational Cost:** Needs to run multiple inference passes.
- **Potential for Overfitting:** If weights are not properly calibrated.

Conclusion

The goal of this project was to investigate and compare various machine learning and

deep learning techniques for the task of facial emotion recognition using the FER2013 dataset. Through comprehensive experimentation across a wide range of models—from traditional classifiers to state-of-the-art neural networks—we have gained meaningful insights into the strengths and limitations of each approach.

The experimental results show that our Custom Weighted Model achieved the highest accuracy at 45.77%, followed closely by traditional machine learning models (SVM, Random Forest, KNN) with accuracies around 45%. Interestingly, the Vision Transformer model performed similarly to traditional approaches with 43.74% accuracy, while CNN and CNN+RNN models showed lower performance than expected.

Our emotion-wise analysis revealed that "Happy" and "Surprise" emotions were consistently recognized with higher accuracy across all models, while emotions like "Fear" and "Disgust" presented more challenges. This pattern highlights the inherent difficulty in distinguishing between certain emotional expressions that may share similar facial features.

The implementation of a real-time web interface using Flask demonstrates the practical application of our emotion recognition system. The modular architecture allows users to select different models for inference, providing flexibility and the ability to compare model performance in real-time scenarios.

From a technical perspective, the combination of OpenCV for face detection, scikit-learn for traditional models, and TensorFlow/Keras for deep learning models created a comprehensive pipeline capable of handling the entire emotion recognition process from image capture to classification.

Future improvements could include:

- Implementation of ensemble methods for better accuracy
- Advanced data augmentation techniques
- Model optimization for improved real-time performance
- Enhanced preprocessing techniques
- Mobile deployment options

In summary, this project successfully demonstrates the application of both traditional

machine learning and modern deep learning techniques in emotion recognition tasks, providing valuable insights into their relative strengths and weaknesses in this challenging computer vision domain.

References

1. FER-2013 Dataset Source: <https://www.kaggle.com/datasets/msambare/fer2013>
2. Project Repository Source:
<https://huggingface.co/kalpisharma/face-emotion-recognition/tree/main>
3. OpenCV: Open Source Computer Vision Library Documentation:
<https://docs.opencv.org/>
4. NumPy: Scientific Computing with Python Documentation: <https://numpy.org/doc/>
5. Pandas: Data Analysis and Manipulation Tool Documentation:
<https://pandas.pydata.org/docs/>
6. Scikit-learn: Machine Learning in Python Documentation:
<https://scikit-learn.org/stable/documentation.html>
7. TensorFlow with Keras API Documentation: https://www.tensorflow.org/api_docs
8. Flask Web Framework Documentation: <https://flask.palletsprojects.com/>
9. Ekman's Six Universal Emotions Theory Overview:
<https://www.paulekman.com/universal-emotions/>
10. Vision Transformer (ViT) Paper: <https://arxiv.org/abs/2010.11929>

